Descrição do processo de gerenciamento: plano de projeto

Artefato 1 do Trabalho Prático de Engenharia de Software (2026.2), projeto okonomi. Rastreado pela issue #25. Atende às instruções 8 a 12 da seção 4 e ao Critério de Avaliação 01 do enunciado.

Versão	Data	Autor	Descrição
1.0	29/09/2026	José Wilson Barbosa de Sousa Júnior	Processo Kanban, quadro e cartões.
1.1	06/11/2026	José Wilson Barbosa de Sousa Júnior	Inventário e capturas do quadro atualizados.
2.0	06/11/2026	José Wilson Barbosa de Sousa Júnior	Reorganizado segundo os modelos de plano de projeto e de plano de iteração; plano e avaliação de cada marco; medições e lições aprendidas.

1. Introdução

Este documento descreve como o projeto okonomi foi gerenciado: a organização da equipe, as práticas de gestão e de engenharia, o quadro e os cartões Kanban, o plano e a avaliação de cada iteração e as lições aprendidas.

Modelos adotados (instrução 3). A estrutura segue o modelo de plano de projeto fornecido (`project_plan_tpl.odt`): Introdução, Organização do projeto, Práticas e medições, Marcos e objetivos, Implantação e Lições aprendidas. Cada marco (iteração) é planejado e avaliado com as seções do modelo de plano de iteração (`iteration_plan_tpl.odt`): datas, objetivos, itens de trabalho, critérios de avaliação e avaliação do planejado em relação ao realizado. O modelo não traz seção própria para o quadro Kanban, exigido pelas instruções 8 a 12; ele é descrito dentro de Práticas (seção 3), que é onde o modelo pede a descrição das práticas de gestão.

2. Organização do projeto

O trabalho é **individual** (a instrução 1 permite de um a cinco participantes). O único membro acumula todos os papéis:

Papel	Responsabilidade	Membro
Dono do produto	Traduz o enunciado em cartões, define critérios de aceitação e prioridade (MoSCoW).	José Wilson Barbosa de Sousa Júnior (matrícula 241024259)
Desenvolvedor	Implementa o protótipo com TDD e escreve os artefatos.	idem
Revisor	Revisa cada pull request contra os critérios de aceitação e a rotina de revisão (ortografia, clareza, consistência, links).	idem
Gestor do processo	Mantém o quadro fiel ao estado real do trabalho e conduz a avaliação de cada marco.	idem

As **áreas de trabalho** são as labels `area:*` dos cartões (autenticação, projetos, backlogs, histórias, épicos, critérios, estimativa, priorização, interface, banco e infraestrutura). Quem construiu cada artefato está em

`ENTREGA.md` (instrução 6).

3. Práticas e medições do projeto

3.1 Método de gerenciamento: Kanban

O projeto é gerenciado pelo método **Kanban**. O trabalho é representado por cartões em um quadro visível a todos; cada cartão atravessa colunas que representam os estados do trabalho, da captura à entrega. O fluxo é **puxado**: ninguém recebe trabalho empurrado; quem está livre puxa o próximo cartão mais prioritário do Sprint Backlog.

O Kanban foi escolhido por três motivos:

- O trabalho é heterogêneo. Documentos (artefatos 1 a 7 e 9), código (protótipo, artefato 8) e infraestrutura convivem no mesmo projeto. Kanban não exige que tudo caiba numa mesma cadência de sprint.
- O gargalo fica visível. Com poucos cartões em execução, uma coluna que acumula cartões aponta onde o fluxo travou.
- O quadro é um registro auditável. Como cada cartão é uma issue do GitHub, o histórico de cada mudança de coluna, comentário e PR fica guardado.

Regras do fluxo:

Regra	Como é aplicada
Limite de WIP	No máximo **1 cartão por pessoa** na coluna Doing. Antes de puxar outro, o atual vai para Review.
Tudo nasce de uma issue	Nenhum commit, branch ou PR existe sem uma issue que lhe dê contexto. Trabalho descoberto no meio do caminho (por exemplo, um defeito) vira uma issue nova antes de ser feito.
Mover na hora	O cartão muda de coluna quando o estado muda, nunca em lote no fim.
Política explícita de saída	Cada coluna tem um critério de saída (seção 3.2). Um cartão só avança quando o cumpre.
Iterações por marco	O trabalho é agrupado em seis marcos (*milestones*), M0 a M5, que funcionam como iterações do processo iterativo e incremental pedido na introdução do enunciado.

3.2 O quadro

Quadro: **okonomi (GitHub Projects)**, na visão *Board*, agrupado pelo campo **Status**.

Coluna	Propósito	Critério de saída
Backlog	Trabalho capturado e priorizado, ainda sem responsável. É a fonte única de tudo que falta fazer.	Escopo entendido, story points e MoSCoW definidos e cartão escolhido para o marco atual.
Sprint Backlog	Trabalho comprometido para a iteração (marco) atual e pronto para ser puxado.	Critérios de aceitação escritos; alguém puxa o cartão, respeitando o limite de WIP.
Doing	Em execução agora. Limite de WIP: 1 cartão por pessoa.	Pull request aberto, com `Closes #N` (ou `Refs #N`) ligando-o à issue.
Review	Pull request aberto, aguardando revisão e o check `ci` da integração contínua.	PR aprovado e `ci` verde.

Coluna	Propósito	Critério de saída
Done	Merge feito no `main`, CI verde e critérios de aceitação verificados.	Evidência registrada na issue (saída de testes, link do PR).

A descrição de cada coluna também está cadastrada no próprio quadro, como descrição da opção do campo Status e no README do projeto no GitHub.

Os nomes *Backlog* e *Sprint Backlog* vêm do vocabulário do Scrum porque é o vocabulário do próprio domínio do sistema (product backlog e sprint backlogs). O processo continua sendo Kanban: não há reuniões nem duração fixa de sprint, o fluxo é puxado e limitado por WIP, e "sprint" aqui significa apenas o marco em andamento.

3.3 Os cartões

Cada cartão do quadro é uma **issue** do repositório WilsonSousajr/okonomi. Pull requests também aparecem no quadro, ligados à issue que fecham.

Informação presente em cada cartão

Campo	Conteúdo	Exemplo
Título	Tipo e resumo; requisitos funcionais começam por `RFnn`, artefatos por `[Artefato n]`.	`RF15: Domínio de story points: 0, 1, 2, 3, 5, 8, 13, 21, 34, 55`
Descrição	Objetivo, escopo e **critérios de aceitação** no formato *Dado … quando … então …*, como caixas de seleção. Issues de artefato trazem também as instruções da seção 4 e os itens do critério de avaliação que cobrem.	ver #15
Label de tipo	`feat` `fix` `docs` `test` `refactor` `chore` `build` `ci` `perf` `style`, o mesmo conjunto dos tipos de commit.	`feat`
Label de área	Parte do sistema afetada: `area:auth`, `area:projects`, `area:backlogs`, `area:stories`, `area:epics`, `area:criteria`, `area:estimation`, `area:prioritization`, `area:ui`, `area:db`, `area:infra`.	`area:estimation`
Label de artefato	`artefato:processo`, `artefato:visao`, `artefato:rnf`, `artefato:historias`, `artefato:arquitetura`, `artefato:interface`, `artefato:bd`, `artefato:prototipo`, `artefato:infraestrutura`. Mapeia 1:1 com os critérios de avaliação.	`artefato:prototipo`
Labels de sinalização	`RF` (implementa um requisito numerado da seção 2), `regression` (defeito que exige teste de regressão), `blocked`, `entrega`.	`RF`
Marco	Iteração em que o cartão será entregue (M0–M5).	`M4: Protótipo`
Story Points	Esforço relativo na escala 0, 1, 2, 3, 5, 8, 13, 21, 34, 55, a mesma que o sistema impõe às histórias (RF15).	`1`
MoSCoW	Prioridade: **M** (*must*), **S** (*should*), **C** (*could*), **W** (*won't*).	`M (Must have)`

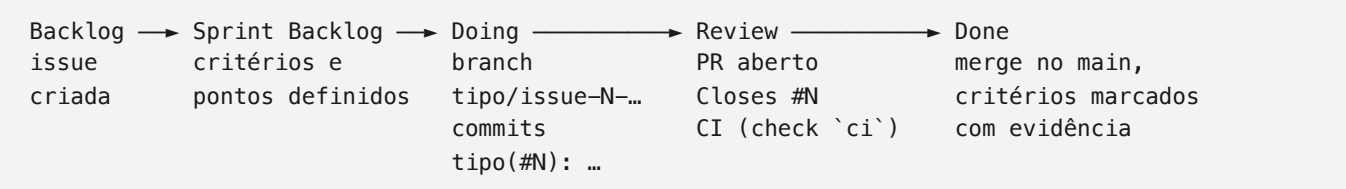
Campo	Conteúdo	Exemplo
RICE	Pontuação (R × I × C) / E, reservada para desempatar cartões de mesma prioridade MoSCoW que disputem a mesma vaga de WIP. Como o projeto tem um único participante e os cartões M de cada marco foram puxados na ordem de dependência entre eles, nenhum empate precisou ser decidido, e o campo ficou vazio em todos os cartões.	vazio
Responsável (*Assignees*)	Quem puxou o cartão. Todos os cartões têm o único membro da equipe como responsável.	`WilsonSousajr`
Status	Coluna atual do quadro.	`Backlog`
PR vinculado	Pull request que implementa o cartão (`Closes #N`).	#54 fecha #24

Três modelos de issue padronizam a criação dos cartões: `requisito-funcional.md`, `artefato.md` e `defeito.md`.

Tipos de cartão

Tipo	Quantidade	Origem
Requisito funcional (`RF`)	19	Um por requisito da seção 2 do enunciado (#1–#19).
Artefato (`artefato:*`)	9	Um por artefato da seção 3 (#25–#33).
Gestão e entrega	3	Processo e equipe (#34), geração dos PDFs (#58) e entrega final (#35).
Fundação e infraestrutura	5	Convenções, esqueleto, banco, CI e modelo de domínio (#20–#24).
Defeito (`regression`)	12	Encontrados durante o trabalho, cada um com teste de regressão (#36, #40, #52, #68, #80–#82, #94, #102–#104, #117).
Manutenção	6	Documentação de apoio e atualização de dependências com avisos de segurança (#46, #48, #66, #85, #108, #114).

3.4 Ciclo de vida de um cartão



1. **Captura.** Uma issue é criada a partir de um modelo, recebe labels, marco, story points e MoSCoW, e entra em **Backlog**.
2. **Compromisso.** No início de um marco, os cartões dele com critérios de aceitação escritos passam para **Sprint Backlog**.
3. **Execução.** Quem puxa o cartão o move para **Doing** e abre a branch `tipo/issue-N-resumo`. Cada commit segue `tipo(#N): mensagem`, validado pelo hook `.githooks/commit-msg`. Código segue TDD: o teste que falha vem antes.
4. **Revisão.** O PR, com título no mesmo padrão e `Closes #N` no corpo, move o cartão para **Review**. A CI roda RuboCop, Brakeman, auditoria de dependências, testes de unidade e testes de sistema; o

`main` é protegido e só aceita merge com o check `ci` verde.

5. **Conclusão.** Após o merge, a issue fecha, o cartão vai para **Done** e os critérios de aceitação são marcados com a evidência.

3.5 Práticas de engenharia

Prática	Como é aplicada
Desenvolvimento iterativo e incremental	Seis marcos (M0–M5), cada um entregando artefatos ou funcionalidades utilizáveis (seção 4).
TDD	O teste que falha vem antes do código. Todo critério de aceitação *Dado … quando … então …* de um cartão `RF` vira um teste.
Teste de regressão para todo defeito	Cada defeito vira um cartão `regression`; o teste que reproduz o defeito falha antes da correção e passa depois.
Integração contínua	GitHub Actions roda RuboCop, Brakeman, auditoria de dependências (gems e JavaScript), testes de unidade e controller e testes de sistema em cada pull request.
`main` protegido	Merge só por pull request e com o check `ci` verde; *force-push* e exclusão recusados.
Padrão de commit e de branch	`tipo(#N): mensagem` e `tipo/issue-N-resumo`, validados pelo hook `.githooks/commit-msg`.
Documentos como código	Os artefatos são Markdown versionado; o PDF é gerado por `bin/docs-pdf` no mesmo pull request.

3.6 Medições

Medida	Como é obtida	Valor em 06/11/2026
Pontos concluídos por marco (velocidade)	Soma do campo Story Points dos cartões fechados de cada marco.	Tabela da seção 4
Escopo planejado × realizado	Cartões do marco na sua abertura × cartões fechados ao fim.	Seção 4.1
Cartões em execução (WIP)	Cartões na coluna Doing.	1 (limite: 1 por pessoa)
Vazão	Pull requests mesclados no `main`.	62 PRs, de 16/09 a 06/11
Defeitos encontrados	Cartões com label `regression`.	12, todos com teste de regressão; 11 corrigidos
Saúde da integração	Check `ci` obrigatório em todo merge.	Nenhum merge sem `ci` verde
Testes automatizados	`bin/rails test` e `bin/rails test:system`.	217 testes de unidade e controller (816 asserções) e 11 arquivos de testes de sistema

4. Marcos e objetivos

Cada marco é uma iteração. Os períodos vão da primeira à última entrega do marco, segundo a data de fechamento dos cartões; M3 e M4 se sobrepuseram de propósito, porque o modelo de dados (M0) já permitia começar o protótipo enquanto a arquitetura era escrita. Cartões de manutenção acrescentados depois (por exemplo, atualizações de segurança) não alongam o período.

Iteração	Objetivos principais	Período	Pontos planejados	Pontos concluídos	Critério de Avaliação
M0: Fundação	Repositório, quadro, esqueleto Rails 8 + PostgreSQL, CI e modelo de domínio com as invariantes do enunciado.	16/09 a 29/09	31	31	nenhum
M1: Gestão do Projeto	Processo Kanban (artefato 1), equipe e autoria, geração de PDF.	17/09 a 29/09	8	8	01
M2: Requisitos	Visão (2), requisitos não funcionais (3) e histórias de usuário (4).	29/09 a 18/10	18	18	02, 03, 04
M3: Projeto	Arquitetura (5), interface (6) e banco de dados (7).	18/10 a 29/10	26	26	05, 06, 07
M4: Protótipo	RF01 a RF19 funcionando de ponta a ponta e vídeo do teste de sistema (8).	18/10 a 02/11	81	79	08
M5: Implantação e Entrega	Infraestrutura (9), auditoria final contra o enunciado e pacote ZIP.	02/11 a 08/11	11	8	09

Os pontos planejados incluem os cartões acrescentados durante o marco (defeitos e atualizações de segurança), descritos na avaliação de cada um.

4.1 Planos e avaliações das iterações

Cada marco segue o modelo de plano de iteração. Os itens de trabalho de cada um estão no inventário (seção 7), filtrando pela coluna Marco; o estado do projeto usa a escala do modelo (verde, amarelo, vermelho).

M0: Fundação

- **Objetivos:** repositório com controle de versões, quadro e convenções; aplicação Rails rodando com PostgreSQL; CI obrigatória; modelo de domínio com as invariantes do enunciado no modelo e no banco.
- **Critérios de avaliação:** `bin/setup` sobe o ambiente do zero; a CI bloqueia merge vermelho; cada invariante tem teste que prova a rejeição do valor inválido.
- **Planejado × realizado:** 5 cartões na abertura (#20–#24), concluídos entre 17/09 e 29/09. Foram acrescentados 8 cartões: 3 defeitos e ajustes achados logo no início (#36, #40, #46) e 5 de manutenção registrados aqui por serem de fundação (#48, #52, #66, #68, #85), o último concluído em 02/11.
- **Avaliação:** verde. Desvio: dois defeitos de ambiente apareceram logo no início (#36, `json 3.0` quebrando os cookies; #52, testes paralelos travando no macOS) e viraram cartões com teste de regressão.

M1: Gestão do Projeto

- **Objetivos:** processo Kanban descrito com quadro, colunas e cartões; equipe e autoria registradas; artefatos textuais em PDF.
- **Critérios de avaliação:** cada item do Critério 01 ligado a uma seção do documento; PDF gerado no mesmo PR do Markdown.

- **Planejado × realizado:** 2 cartões planejados e 1 acrescentado (#58, geração de PDF, quando a instrução 7 foi revista); os 3 concluídos em 29/09.
- **Avaliação:** verde.

M2: Requisitos

- **Objetivos:** visão e escopo, requisitos não funcionais com normas e métricas, e as 19 histórias de usuário no formato *Como … eu quero … para …*.
- **Critérios de avaliação:** cada requisito da seção 2 do enunciado coberto por uma história; cada RNF verificável.
- **Planejado × realizado:** 3 de 3 cartões, entre 29/09 e 18/10.
- **Avaliação:** verde. Desvio: a auditoria de 04/11 encontrou números divergentes entre visão e RNF (largura mínima de tela, desempenho) e critérios escritos do ponto de vista do desenvolvedor; corrigidos no PR #116.

M3: Projeto

- **Objetivos:** architecture notebook, storyboards com wireframes e projeto físico do banco.
- **Critérios de avaliação:** documentos coerentes com o código (rotas, tabelas, colunas e chaves conferidas com `db/schema.rb`).
- **Planejado × realizado:** 3 de 3 cartões, entre 18/10 e 29/10.
- **Avaliação:** verde. Desvio: a auditoria de 04/11 encontrou rotas desatualizadas na arquitetura e serviços sem storyboard (redefinição de senha, exclusões); corrigidos no PR #119, com dois storyboards novos.

M4: Protótipo

- **Objetivos:** os 19 requisitos funcionais implementados e testados; vídeo com um cenário de sucesso para cada serviço.
- **Critérios de avaliação:** todo critério de aceitação com teste verde; fluxo exercitado no navegador; vídeo gerado pelo teste de sistema.
- **Planejado × realizado:** 20 cartões planejados (RF01–RF19 e o vídeo) e 7 defeitos acrescentados ao longo do marco (#80–#82, #94, #102–#104); 26 de 27 concluídos (79 de 81 pontos). Fica aberto o #94 (testes de sistema intermitentes no CI), que não afeta o protótipo.
- **Avaliação:** amarelo, por causa do #94. A instabilidade faz o CI precisar de nova execução às vezes; o vídeo e o pacote de entrega não dependem dela.

M5: Implantação e Entrega

- **Objetivos:** infraestrutura de implantação; auditoria de todos os artefatos contra o enunciado; pacote `ESW-241024259.ZIP`.
- **Critérios de avaliação:** cada item dos nove critérios de avaliação atendido e rastreado; pacote descompacta, passa nos testes e no antivírus.
- **Planejado × realizado:** 2 cartões planejados e 3 acrescentados (#108 e #114, avisos de segurança em dependências; #117, páginas de erro em inglês); 4 de 5 concluídos. O #35 (entrega final) fecha com o envio, até 08/11.
- **Avaliação:** verde. A auditoria de 04/11 gerou os PRs #112, #113, #115, #116, #118, #119 e #122.

4.2 Implantação

A estratégia de implantação (servidor, serviços, Kamal e procedimento passo a passo) está no artefato 9. A entrega à disciplina é o pacote ZIP descrito em `ENTREGA.md` e rastreado no cartão #35.

5. Atividades de gerenciamento realizadas

Atividade	Evidência
Criação do repositório com controle de versões (instrução 2).	WilsonSousajr/okonomi
Criação do quadro Kanban com cinco colunas e descrição de cada uma (instruções 9 e 10).	Projeto 14
Criação dos campos de cartão Story Points, MoSCoW e RICE.	Campos do projeto
Decomposição do enunciado em cartões: 19 RF, 9 artefatos, gestão e entrega (instrução 11).	Issues #1–#35
Rastreamento de cada instrução (seção 4 do enunciado) e de cada item de critério (seção 5 do enunciado) como caixa de seleção dentro de um cartão.	Issues de artefato, #34 e #35
Criação de labels de tipo, área, artefato e sinalização, e dos marcos M0–M5.	Labels, Marcos
Estimativa (story points) e priorização (MoSCoW) de todos os cartões.	Campos no quadro; inventário na seção 7
Definição do padrão de commit e de branch, com validação automática.	`.githooks/commit-msg`, `AGENTS.md`
Proteção do `main`: PR obrigatório, check `ci` obrigatório, sem *force-push*.	Regras da branch
Integração contínua com análise estática e testes.	`.github/workflows/ci.yml`
Registro de defeitos encontrados como cartões com teste de regressão.	#36, #40, #52, #68, #80–#82, #94, #102–#104, #117
Geração dos artefatos textuais em PDF a partir do Markdown (instrução 7).	`bin/docs-pdf`, `docs/pdf/`, #58
Atualização de dependências com avisos de segurança, cada uma em cartão próprio antes de qualquer outro merge.	#68, #85, #108, #114
Auditoria de todos os artefatos contra o enunciado (04/11/2026): cada item dos critérios de avaliação conferido com o documento e com o código, e as divergências corrigidas em PRs próprios.	PRs #112, #113, #115, #116, #118, #119
Revisão do quadro (06/11/2026): todos os PRs incluídos, cartões fechados movidos para Done, responsável atribuído e estimativa preenchida nos cartões que não tinham.	Projeto 14

6. Lições aprendidas

Lição	Ação tomada ou recomendada
O quadro desatualiza se só for revisto no fim. O inventário e as capturas ficaram de 29/09 a 06/11 sem atualização, PRs não entraram no quadro e cartões fechados ficaram em Review.	Revisão do quadro como passo obrigatório da avaliação de cada marco, e não só da entrega.
Documento e código divergem em silêncio. Versões de gems, rotas e o `config/deploy.yml` descritos nos documentos já não batiam com o repositório.	Auditoria dos artefatos contra o enunciado e contra o código ao fim de cada marco; versões citadas "conforme o `Gemfile.lock`" com data.

Lição	Ação tomada ou recomendada
Avisos de segurança chegam sem aviso e travam todos os PRs. Isso aconteceu quatro vezes (#68, #85, #108, #114).	Atualização da dependência em PR próprio, antes de qualquer outro merge.
Testes de sistema instáveis custam tempo de merge. O #94 exigiu de uma a três novas execuções por PR na última semana.	Próximo passo registrado no #94: esperar a pintura da página antes de cada ação ou trocar o *driver* (Cuprite).
A regra "todo defeito vira teste" funcionou. Os 12 defeitos encontrados têm teste de regressão, e nenhum voltou.	Manter a regra.
O RICE no quadro só é útil com disputa por WIP. Com um único participante e a ordem ditada pelas dependências, o campo nunca foi necessário.	Em equipe, preencher o RICE ao montar o Sprint Backlog de cada marco.

7. Inventário de cartões

Retrato do quadro em **06/11/2026**, gerado pela API do GitHub Projects (`gh api graphql`, campos Status, Story Points e MoSCoW de cada item). O estado atual está sempre no quadro. Todos os cartões têm o único membro da equipe como responsável.

Cartão	Título	Labels	Marco	Pontos	MoSCoW	Coluna
#1	RF01: Interface gráfica web para acesso ao sistema	`feat` `area:ui` `RF` `artefato:prototipo`	M4	5	M	Done
#2	RF02: Criação de conta de usuário	`feat` `area:auth` `RF` `artefato:prototipo`	M4	3	M	Done
#3	RF03: Autenticação e acesso aos serviços	`feat` `area:auth` `RF` `artefato:prototipo`	M4	3	M	Done
#4	RF04: CRUD de projetos	`feat` `area:projects` `RF` `artefato:prototipo`	M4	3	M	Done
#5	RF05: CRUD de product backlog (no máximo um por projeto)	`feat` `area:backlogs` `RF` `artefato:prototipo`	M4	5	M	Done
#6	RF06: CRUD de sprint backlogs (vários por projeto)	`feat` `area:backlogs` `RF` `artefato:prototipo`	M4	3	M	Done
#7	RF07: CRUD de histórias de usuário associadas ao product backlog	`feat` `area:stories` `RF` `artefato:prototipo`	M4	5	M	Done
#8	RF08: Movimentação de histórias de usuário entre backlogs	`feat` `area:stories` `RF` `artefato:prototipo`	M4	5	M	Done
#9	RF09: Formato padrão da história: Como um … eu quero … para …	`feat` `area:stories` `RF` `artefato:historias`	M4	2	M	Done
#10	RF10: CRUD de épicos	`feat` `area:epics` `RF` `artefato:prototipo`	M4	3	M	Done
#11	RF11: Vinculação de histórias de usuário a épicos	`feat` `area:epics` `RF` `artefato:prototipo`	M4	3	M	Done

Cartão	Título	Labels	Marco	Pontos	MoSCoW	Coluna
#12	RF12: CRUD de critérios de aceitação	`feat` `area:criteria` `RF` `artefato:prototipo`	M4	3	M	Done
#13	RF13: Formato padrão do critério: Dado … quando … então …	`feat` `area:criteria` `RF` `artefato:historias`	M4	2	M	Done
#14	RF14: Atribuição de story points a histórias de usuário	`feat` `area:estimation` `RF` `artefato:prototipo`	M4	2	M	Done
#15	RF15: Domínio de story points: 0, 1, 2, 3, 5, 8, 13, 21, 34, 55	`feat` `area:estimation` `RF` `artefato:prototipo`	M4	1	M	Done
#16	RF16: Atribuição de etiqueta MoSCoW (M, S, C, W)	`feat` `area:prioritization` `RF` `artefato:prototipo`	M4	2	M	Done
#17	RF17: Atribuição de critério RICE (Reach, Impact, Confidence, Effort)	`feat` `area:prioritization` `RF` `artefato:prototipo`	M4	3	M	Done
#18	RF18: Domínios dos valores RICE	`feat` `area:prioritization` `RF` `artefato:prototipo`	M4	2	M	Done
#19	RF19: Cálculo da pontuação RICE pela fórmula (R × I × C) / E	`feat` `area:prioritization` `RF` `artefato:prototipo`	M4	3	M	Done
#20	chore: diretrizes do projeto e convenções de contribuição	`docs` `chore`	M0	3	M	Done
#21	chore: esqueleto Rails 8.1 com PostgreSQL e Tailwind	`chore` `area:infra`	M0	5	M	Done
#22	build: PostgreSQL de desenvolvimento via Docker Compose	`build` `area:db` `area:infra`	M0	2	M	Done
#23	ci: pipeline de integração contínua	`ci` `area:infra`	M0	3	S	Done
#24	feat: modelagem base do domínio (migrations, models e invariantes)	`feat` `area:db`	M0	8	M	Done
#25	[Artefato 1] Descrição do processo: quadro e cartões Kanban	`docs` `artefato:processo`	M1	3	M	Done
#26	[Artefato 2] Documento de visão e escopo	`docs` `artefato:visao`	M2	5	M	Done
#27	[Artefato 3] Especificação de requisitos não funcionais	`docs` `artefato:rnf`	M2	8	M	Done
#28	[Artefato 4] Especificação de requisitos funcionais por histórias de usuário	`docs` `artefato:historias`	M2	5	M	Done
#29	[Artefato 5] Architecture notebook	`docs` `artefato:arquitetura`	M3	8	M	Done
#30	[Artefato 6] Projeto de interface: storyboards e wireframes	`docs` `area:ui` `artefato:interface`	M3	13	M	Done

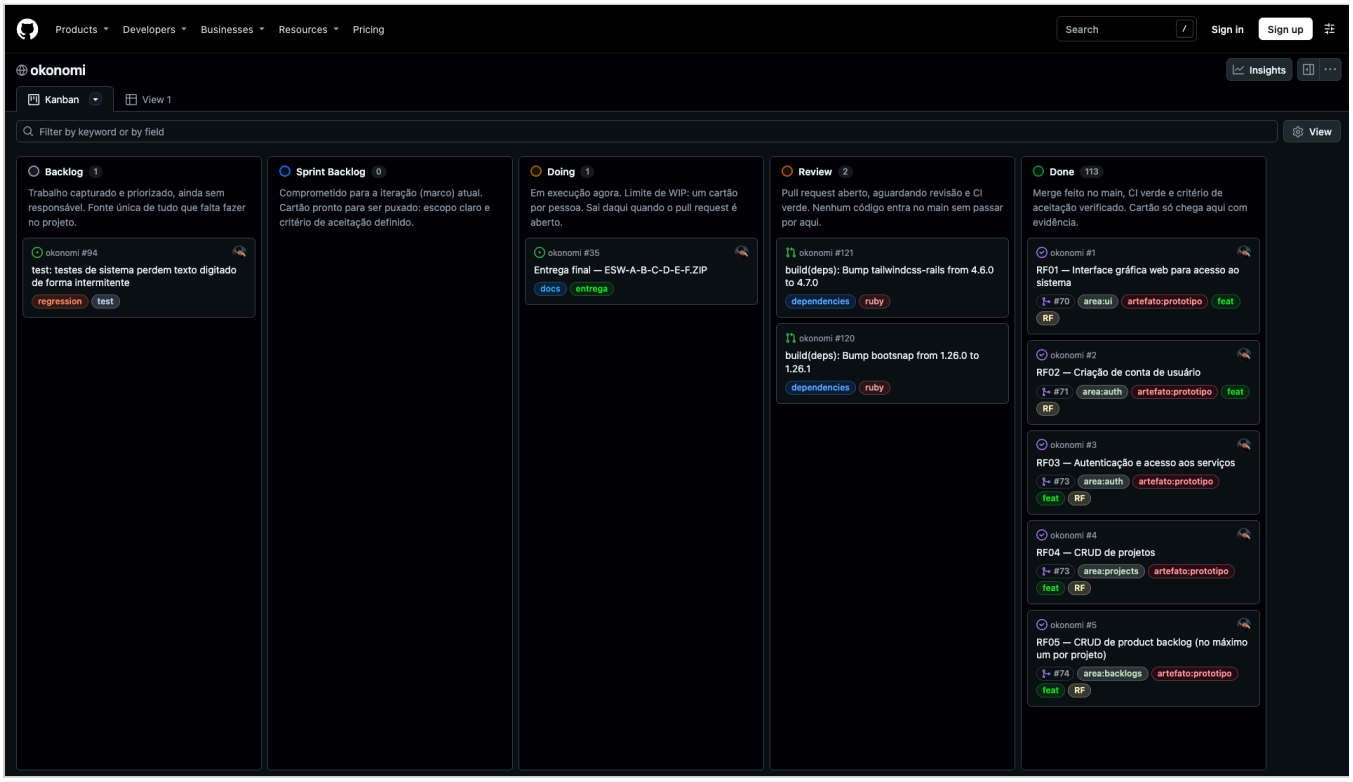
Cartão	Título	Labels	Marco	Pontos	MoSCoW	Coluna
#31	[Artefato 7] Projeto físico do banco de dados	`docs` `area:db` `artefato:bd`	M3	5	M	Done
#32	[Artefato 8] Protótipo e vídeo de teste de sistema	`docs` `artefato:prototipo`	M4	13	M	Done
#33	[Artefato 9] Descrição da infraestrutura de implantação	`docs` `area:infra` `artefato:infraestrutura`	M5	5	M	Done
#34	Processo e equipe: participantes, templates e revisão dos artefatos	`docs` `artefato:processo`	M1	2	M	Done
#35	Entrega final: ESW-A-B-C-D-E-F.ZIP	`docs` `entrega`	M5	3	M	Doing
#36	fix: json 3.0 quebra a leitura de cookies assinados (autenticação inteira falha)	`fix` `build` `area:auth` `regression`	M0	2	M	Done
#40	fix: cookie de sessão sem flag secure e force_ssl desativado em produção	`fix` `area:auth` `area:infra` `regression`	M0	2	M	Done
#46	docs: CLAUDE.md duplica regras que pertencem só ao AGENTS.md	`docs`	M0	1	M	Done
#48	build: ignorar o json no Dependabot enquanto a fixação em 2.x for necessária	`build` `ci`	M0	1	M	Done
#52	fix: suíte de testes trava no macOS quando roda em paralelo	`fix` `area:infra` `regression`	M0	1	M	Done
#58	build: gerar os artefatos textuais em PDF	`docs` `build` `entrega`	M1	3	M	Done
#66	docs: documentar o uso do ai-memory no AGENTS.md	`docs`	M0	1	C	Done
#68	fix: CI falha no Brakeman porque a versão 8.0.6 não é a mais recente	`fix` `ci` `regression`	M0	1	M	Done
#80	fix: senha curta na redefinição aparece como "As senhas não conferem"	`fix` `area:auth` `regression`	M4	1	M	Done
#81	fix: senha acima de 72 bytes mostra "Translation missing"	`fix` `area:auth` `regression`	M4	1	M	Done
#82	fix: entrar depois de uma sessão expirada pode levar a uma página 404	`fix` `area:auth` `regression`	M4	2	M	Done
#85	chore: atualizar net-ssh e crass (avisos do bundler-audit quebram o CI)	`chore`	M0	1	M	Done

Cartão	Título	Labels	Marco	Pontos	MoSCoW	Coluna
#94	test: testes de sistema perdem texto digitado de forma intermitente	`test` `regression`	M4	2	S	Backlog
#102	fix: story points, Confidence e Effort aceitam valores não inteiros truncando-os em silêncio	`fix` `area:estimation` `regression`	M4	2	M	Done
#103	fix: épico excluído ou inexistente no formulário da história gera erro 500	`fix` `area:epics` `regression`	M4	1	M	Done
#104	fix: Reach acima do limite do inteiro gera erro 500	`fix` `area:prioritization` `regression`	M4	1	M	Done
#108	chore: atualizar addressable, loofah, nokogiri, rack e rails-html-sanitizer (avisos do bundler-audit quebram o CI)	`chore`	M5	1	M	Done
#114	chore: atualizar o Rails para 8.1.4.1 (avisos de segurança do bundler-audit quebram o CI)	`chore`	M5	1	M	Done
#117	fix: páginas de erro (404, 422, 500…) em inglês violam RNF-08	`fix` `area:ui` `regression`	M5	1	S	Done

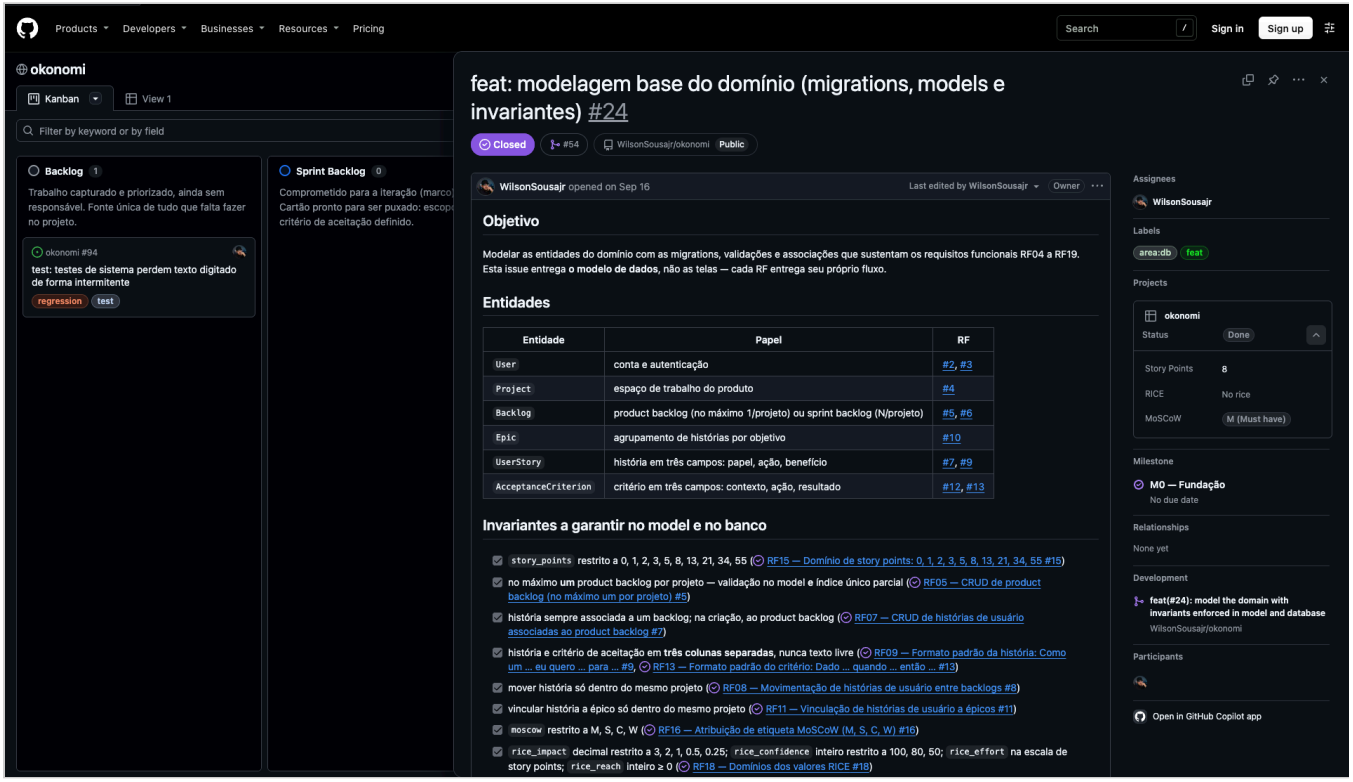
Pull requests também aparecem no quadro, ligados à issue que fecham: ficam em Review enquanto abertos e vão para Done após o merge. Foram omitidos do inventário para não duplicar cartões. PRs fechados sem merge (por exemplo, atualizações do Dependabot descartadas) não ficam no quadro.

8. Evidência

Visão *Board* do quadro, com as cinco colunas:



Detalhe de um cartão (#24), com responsável, labels, coluna (Status), Story Points, MoSCoW, marco e PR vinculado:



9. Rastreabilidade com o enunciado

Enunciado	Seção deste documento
Instrução 3: adotar modelos	1
Instrução 8: adotar método Kanban	3.1
Instrução 9: quadro com colunas apropriadas	3.2, 8
Instrução 10: propósito de cada coluna	3.2
Instrução 11: criar cartões	3.3, 7
Instrução 12: informação sobre cada cartão	3.3, 7
Critério 01: atividades de gerenciamento realizadas	3.4, 4.1, 5
Critério 01: método Kanban adotado	3.1
Critério 01: quadro com colunas apropriadas	3.2, 8
Critério 01: propósito de cada coluna	3.2
Critério 01: cartões criados	3.3, 7, 8
Critério 01: informação sobre cada cartão	3.3, 7